

Eventology V1 — MVP End-to-End Test Plan

Purpose. A complete, per-module manual e2e script covering every business workflow, rule, and decision in the MVP. You test **one section at a time**, record results inline, and send them back. Anything that needs a fix / feature / refactor becomes a **mesh handoff**; once merged we move to the next section.

MVP constraints (apply to the whole document):

- **Payments are OFF** (`NEXT_PUBLIC_PAYMENTS_ENABLED=false`). Only **free** tickets register; revenue/payout/commission UIs are placeholders; Chapa is unreachable. Paid code paths still exist behind the flag.
- **AI is stubbed** (`AI_PROVIDER=stub`) — deterministic fixtures, **advisory + fail-open** (never blocks a user action).
- **Email / SMS / Push are stubbed** — no real delivery. You verify the in-app record/intent, not the external send.
- RLS is the source of truth **and** every admin/organizer route also enforces an app-level role/ownership guard.

0. Environments, accounts & how to record results

Targets

Surface	URL	Notes
Web — Production	<code>https://eventology-nu.vercel.app</code>	live, <code>570c36a</code> , payments OFF
Web — Local	<code>http://localhost:3000</code>	<code>npm run dev</code> (web); reads <code>apps/web/.env.local</code>
Mobile — Expo Go	LAN: <code>http://192.168.1.42:3000</code>	set in <code>EXPO_PUBLIC_API_URL</code> ; confirm your PC's current LAN IP + port , start web with <code>next dev -H 0.0.0.0</code> , phone on same Wi-Fi

Decision — test order. Do **local first** (faster hot-reload for fixes), then re-run the critical paths against production. Mobile runs on your **physical S25 Ultra** via Expo Go (the QR check-in scanner needs a real camera).

Test accounts you need (create or seed)

- **A1 — Attendee** (default role on signup).
- **O1 — Organizer (verified)** — to test the happy organizer path.
- **O2 — Organizer (unverified / pending)** — to test the verification gate.
- **AD1 — Admin** — role must be set to `admin` (DB or admin tooling; signups default to `attendee`).

Roles in the system: `attendee` · `organizer` · `admin`. New signups are `attendee`. Becoming an organizer creates an `organizers` row with `verification_status pending` → `verified/rejected`.

Seed data already live

55 events · 150 registrations · 150 tickets · 10 organizers · 12 venues · 8 categories · 5 sponsors. Events span statuses (draft/pending/approved/rejected/cancelled), free + paid tiers, verified + unverified organizers, at least one with sponsors + a conversation.

How to record (copy this per scenario)

Each scenario row has an **Expected**. Mark **Result** =  PASS /  FAIL /  PARTIAL, add **Notes** (what you saw, screenshot ref, console/network error). Send the section's filled table back. I triage: **P0** (blocks the flow) → immediate mesh handoff; **P1/P2** → batched handoff or fold-forward.

1. Cross-cutting rules (verify once, assume thereafter)

#	Rule	Expected behavior	Result	Notes
X1	Auth required	Hitting any <code>/api/protected/*</code> or <code>org/admin</code> page while logged out → redirect to login / <code>401 UNAUTHORIZED</code>		
X2	Role gate (app-level)	Attendee opening <code>/org/*</code> or <code>/admin/*</code> → blocked (redirect/403), not just hidden links		
X3	Ownership gate	Organizer O1 cannot read/edit O2's event/team/sponsors/analytics (404 or 403, never another org's data)		
X4	Error envelope	API errors return <code>{ error: { code, message } }</code> ; UI shows a friendly message, never a raw stack		
X5	Loading / empty / error states	Every list/detail shows a skeleton while loading, a proper empty state when no data, and an error state on failure		
X6	Payments OFF	Nowhere in the app can you reach a checkout / pay screen; no price → payment transition		
X7	AI fail-open	With AI stubbed, no user action is ever blocked by AI; AI panels show advisory output or degrade silently		
X8	i18n	Switching EN↔AM changes copy app-wide and persists across reloads (cookie + localStorage)		

2. Module: Auth & Accounts

Workflows: sign up, log in, log out, session persistence, become-organizer.

Rules/decisions:

- better-auth; `session.user.id` is the profile UUID. New users = role `attendee`.
- Authed Supabase access is bridged via a minted **HS256 JWT** (legacy secret) so `auth.uid()` resolves for RLS.  **This is the #1 risk to validate first** — if the project's legacy JWT secret was revoked in favor of the new ECC signing key, authed reads will 401.
- Becoming an organizer creates an `organizers` row at `verification_status='pending'`.

#	Scenario	Steps	Expected	Result	Notes
2.1	Sign up	Register A1 with email/password	Account created, logged in, role attendee, lands on home/dashboard		
2.2	JWT bridge (CRITICAL)	As A1, open "My Tickets" / any page that reads the user's own rows	Data loads (no 401/permission error in network tab) → confirms the JWT→RLS bridge works		
2.3	Log out / back in	Log out, log back in	Session cleared then restored; protected pages reachable again		
2.4	Session persistence	Refresh while logged in	Stays logged in		
2.5	Become organizer	A1 → "become organizer" / create organizer profile	organizers row created, status pending ; organizer area appears but gated where verification is required		
2.6	Wrong password / dup email	Bad login; signup with existing email	Clear validation errors, no crash		

3. Module: Public Discovery (home, search, categories, event detail)

Workflows: browse home, featured/recommendations, search + filters, category browse, event detail.

Rules/decisions:

- **Only approved events are publicly visible** (anon read). Draft/pending/rejected/cancelled never show publicly.
- Anon reads go through the **anon** Supabase client → public data only; PII (registrations, payments, push tokens, audit log) returns 0 rows to anon.
- Recommendations rail is AI-assisted (stubbed → deterministic) and must render regardless.

#	Scenario	Expected	Result	Notes
3.1	Home loads	Hero, featured carousel, category grid, recommendations rail all render with real seeded events		
3.2	Only approved shown	No draft/pending/rejected event appears anywhere public		
3.3	Search by keyword	Returns matching approved events; empty state for nonsense query		
3.4	Filters	Category / date / type filters narrow results correctly		
3.5	Category browse	Each category page lists its approved events		

#	Scenario	Expected	Result	Notes
3.6	Event detail	Title, banner, description, venue, organizer, tiers all correct; banner image loads from storage		
3.7	Payments-OFF on detail	Paid tiers show disabled "Tickets on sale soon" / no "From X" price CTA; free tier shows a working register CTA		
3.8	Anon PII lock	(Optional, devtools) hitting a protected data path while logged out returns no rows / 401		

4. Module: Registration (free path; payments-off)

Workflows: register for a free event; attempt a paid tier (should be blocked); capacity & duplicate handling.

Business rules (verified in `create_registration_atomic`):

- Event **must be approved** → else `EVENT_NOT_AVAILABLE`.
- **Capacity:** tier `capacity = 0` means unlimited; if `sold_count >= capacity` → `SOLD_OUT` (409). Atomic row-lock prevents oversell.
- **Duplicate:** an existing non-cancelled registration for the same event → `ALREADY_REGISTERED` (409).
- **Free tier** (`price = 0`) → registration `status='confirmed'` and a **ticket is issued immediately** (`TKT-XXXXXXX`).
- **Paid tier with payments OFF** → server returns `503 PAYMENTS_DISABLED`; UI must prevent reaching this.
- Server also re-checks tier→event match (`TIER_EVENT_MISMATCH`).

#	Scenario	Expected	Result	Notes
4.1	Register free event	A1 registers a free, approved event	Success; registration <code>confirmed</code> ; ticket issued and visible in My Tickets	
4.2	Confirmation feedback	After 4.1	Success message (i18n <code>registration.successFree</code>); no payment redirect	
4.3	Duplicate guard	A1 registers the same event again	Blocked with "already registered" (409), no second ticket	
4.4	Sold-out	Register a tier at capacity (seed/find one)	"Sold out" (409), no registration created, <code>sold_count</code> unchanged	
4.5	Paid tier blocked (UI)	Try to register a paid tier	Tier disabled / "Soon"; cannot submit	
4.6	Paid tier blocked (API)	(devtools) POST a paid <code>ticket_tier_id</code> to <code>/api/protected/registrations</code>	<code>503 PAYMENTS_DISABLED</code> ; no payment row created	
4.7	Non-approved	Try to register a draft/pending event (direct URL)	<code>EVENT_NOT_AVAILABLE</code> ; blocked	

#	Scenario	Expected	Result	Notes
	event			

5. Module: Tickets (issuance, QR, My Tickets, offline)

Workflows: view my tickets, open a ticket QR, mobile offline cache.

Rules: ticket **qr_data** is signed; verification is **server-side HMAC only** (clients render, never verify). Ticket number = **TKT-** + first 8 of registration id (upper).

#	Scenario	Expected	Result	Notes
5.1	My Tickets list	A1 sees their confirmed tickets with event info		
5.2	Ticket detail / QR	Opening a ticket shows a scannable QR + ticket number + tier		
5.3	Status reflects check-in	After being checked in (§8), ticket shows used		
5.4	Mobile offline	(mobile) load tickets online, enable airplane mode, reopen	Cached tickets + QR still render (AsyncStorage)	

6. Module: Organizer — Events (create, draft, submit, edit)

Workflows: create event (draft), edit, add tiers, submit for review, preview.

Rules/decisions:

- New events start **draft**. Organizer submits → **pending**. Admin approves → **approved** (public) or **rejected**.
- Only **verified** organizers should be able to publish/submit (verify the gate).
- Banner upload → **event-banners** storage bucket (writes scoped to the user's own folder).
- Event **ticket_type** free/paid; tiers have price, capacity, currency.

#	Scenario	Expected	Result	Notes
6.1	Create draft	O1 creates an event	Saved as draft , visible only to O1, not public	
6.2	Add tiers	Add a free tier (+ a paid tier for later)	Tiers persist; capacity/price validated	
6.3	Banner upload	Upload a banner image	Uploads to storage, renders on the event; another user can't overwrite it	
6.4	Preview	Preview the draft	Renders the public layout without publishing	
6.5	Submit for review	Submit draft	Status → pending ; appears in admin moderation queue	

#	Scenario	Expected	Result	Notes
6.6	Edit	Edit title/description/tiers	Changes persist; editing rules respected by status	
6.7	Verification gate	O2 (unverified) tries to submit/publish	Blocked or clearly gated with reason	
6.8	Ownership	O1 cannot open/edit O2's event via direct URL	403/404	

7. Module: Organizer — Team

Workflows: invite team member, list members, role within org, remove.

Rules: organizer **owner** manages the team; members read. Server enforces org ownership (admin bypass).

#	Scenario	Expected	Result	Notes
7.1	Invite member	O1 invites by email	Invite created; appears in team list	
7.2	Permissions	A non-owner member cannot manage the team	Manage actions hidden + server-rejected	
7.3	Cross-org	O1 cannot view O2's team	403/404	
7.4	Remove	Owner removes a member	Member gone; loses access	

8. Module: Organizer — Check-in (web scanner + mobile scanner)

Workflows: scan a ticket QR at the door; handle re-scan and invalid codes.

Business rules: POST scanned payload to `/api/protected/check-in`; **HMAC verified server-side**.

Outcomes: **success** (ticket → **used**), **ALREADY_CHECKED_IN**, **INVALID_QR** / **TICKET_NOT_FOUND**. Only the event's organizer (or admin) may check in its tickets.

#	Scenario	Expected	Result	Notes
8.1	Web scan (valid)	Web check-in page, scan/enter a valid ticket	Success; running count +1; ticket → used	
8.2	Re-scan	Scan the same ticket again	ALREADY_CHECKED_IN state, count not double-incremented	
8.3	Invalid	Scan a random / tampered payload	INVALID_QR / TICKET_NOT_FOUND	
8.4	Mobile scan	(S25 Ultra) organizer check-in scanner, camera permission, scan a ticket	Same outcomes as web; cooldown prevents rapid double-fire	

#	Scenario	Expected	Result	Notes
8.5	Manual fallback	Enter the payload manually on mobile	Works identically	
8.6	Cross-org	O1 scans a ticket for O2's event	Rejected (ownership)	

9. Module: Organizer — Messaging & Broadcast

Workflows: organizer↔attendee conversation; organizer broadcast to event attendees.

Rules/decisions:

- Conversations: only participants read/write (RLS); **sender_id** forced to the caller.
- **Broadcast** is rate-limited **5/hour per organizer** (in-memory — per server instance), **fail-open** fan-out (one bad recipient doesn't fail the batch), and **excludes the organizer** from recipients.

#	Scenario	Expected	Result	Notes
9.1	Start/continue conversation	Message between O1 and an attendee	Both see the thread; non-participants cannot	
9.2	Broadcast	O1 broadcasts to an event's attendees	All attendees get a notification; organizer excluded	
9.3	Rate limit	Send >5 broadcasts in an hour	6th is rate-limited with a clear message	
9.4	Fail-open	(best-effort) broadcast with a mix of recipients	Batch completes; no hard failure surfaced to organizer	

10. Module: Organizer — Analytics

Workflows: per-event analytics (web + mobile): registrations, views, conversion, tier & city distribution, 30-day trend.

Rules: organizer ownership enforced; reads via `/organizers/[id]/stats` + `/events/[eventId]/analytics`.

#	Scenario	Expected	Result	Notes
10.1	Web analytics	O1 opens an event's analytics	Numbers match seed/registrations; charts render	
10.2	Mobile analytics	(S25) organizer analytics screen	Top-line stats + 30-day bars + tier/sub-city distribution render	
10.3	Ownership	O1 cannot see O2's analytics	403/404	
10.4	Empty event	Analytics for a 0-registration event	Clean zero-state, no NaN/divide-by-zero	

11. Module: Organizer — Revenue (payments OFF)

#	Scenario	Expected	Result	Notes
11.1	Revenue page	<code>/org/revenue</code>	"Payments coming soon" placeholder; no live numbers	
11.2	Nav	Organizer sidebar	No "Revenue" link (removed while OFF)	
11.3	No payout actions	Anywhere in org area	No request-payout / earnings actions reachable	

12. Module: Admin — Dashboard & AI (moderation / fraud / health)

Workflows: admin dashboard overview; AI moderation/fraud/health panels.

Rules/decisions: all admin routes behind `requireAdminRoute` + audit logging. **AI is advisory + fail-open** — it never auto-blocks; it surfaces signals an admin acts on. Stubbed AI returns deterministic fixtures.

#	Scenario	Expected	Result	Notes
12.1	Admin gate	AD1 reaches <code>/admin</code> ; A1/O1 cannot	Admin-only access enforced server-side	
12.2	Dashboard	Overview cards/counts render from live data		
12.3	AI panels	Moderation/fraud/health panels render (stub fixtures), no crash if AI "unavailable"		
12.4	Advisory only	AI flags never auto-block an event/user; admin must act		
12.5	Audit logging	Admin actions write an <code>audit_log</code> row		

13. Module: Admin — Event Approval (moderation queue)

Workflows: review pending events → approve / reject (with reason).

Rules: `pending` → `approved` (becomes public) or `pending` → `rejected` (with a reason). Reject reason captured via dialog (not a raw prompt).

#	Scenario	Expected	Result	Notes
13.1	Queue	Pending events (incl. the one O1 submitted in §6) listed		
13.2	Approve	Approve an event	Status → <code>approved</code> ; now public; organizer notified (in-app)	
13.3	Reject + reason	Reject with a reason	Status → <code>rejected</code> ; reason stored + shown to organizer; not public	
13.4	Audit	Approve/reject writes <code>audit_log</code>		

14. Module: Admin — Organizer Verification

Rules: organizer `verification_status` `pending` → `verified` / `rejected`. Verified status gates organizer publishing (\$6.7).

#	Scenario	Expected	Result	Notes
14.1	Verify O2	Admin verifies the pending organizer	Status → <code>verified</code> ; O2 can now publish	
14.2	Reject	Reject an organizer	Status → <code>rejected</code> ; publish stays gated	

15. Module: Admin — Users & Reviews

#	Scenario	Expected	Result	Notes
15.1	Users list	Admin lists users with roles		
15.2	Role change (if supported)	Promote/demote a user	Reflected in access; audit logged	
15.3	Reviews moderation	Admin reviews list + moderate/remove	Action persists; route behind admin guard	

16. Module: Admin — Revenue (OFF) & Audit Log

#	Scenario	Expected	Result	Notes
16.1	Admin revenue	<code>/admin/revenue</code>	"Payments coming soon" placeholder; no live numbers	
16.2	Nav	Admin sidebar	No "Revenue" link	
16.3	Audit log	Audit log view lists recent admin actions		

17. Module: Notifications & Settings

Rules: in-app notifications work; external channels (email/SMS/push) are **stubbed** (verify intent/record, not delivery). Notification preferences are per-profile.

#	Scenario	Expected	Result	Notes
17.1	In-app notifications	Registration confirmed / ticket issued / broadcast / approval generate in-app notifications		
17.2	Preferences	Toggle notification preferences	Persists per user	

#	Scenario	Expected	Result	Notes
17.3	Stubbed channels	No real email/SMS/push sent	Expected (deferred); confirm no errors thrown	

18. Module: i18n (EN / AM)

#	Scenario	Expected	Result	Notes
18.1	Switch language	Toggle EN↔AM in navbar	Copy changes on main flows (nav, register, event detail, tickets)	
18.2	Persistence	Reload after switching	Language sticks (cookie + localStorage), SSR matches	
18.3	Amharic completeness	Browse main flows in AM	No raw English keys leak on primary paths (long-tail event-detail literals may remain EN — known, log them)	
18.4	Formatting	Dates/currency in AM	Locale-aware formatting	

19. Module: Mobile App (Expo Go — attendee + organizer)

Setup: `EXPO_PUBLIC_API_URL` = your PC LAN IP:port, `next dev -H 0.0.0.0`, Node 22 LTS, Expo Go on the S25 Ultra. Payments OFF mirror (`EXPO_PUBLIC_PAYMENTS_ENABLED=false`).

#	Scenario	Expected	Result	Notes
19.1	Boot + auth	App boots, log in as A1	Session works against the web API over LAN	
19.2	Discover/search/detail	Browse + open an event	Parity with web; only approved events	
19.3	Register (free)	Register a free event on mobile	Confirmed + ticket; paid tiers disabled	
19.4	My Tickets + offline	View tickets, airplane mode	Cached QR renders offline	
19.5	Organizer area	Log in as O1 → organizer event list	Lists O1's events + counts	
19.6	Mobile check-in	Scan a ticket (camera)	Success / already / invalid states (see §8)	
19.7	Mobile analytics	Open per-event analytics	Renders correctly	

20. Security & RLS spot-checks (authz boundaries)

#	Scenario	Expected	Result	Notes
20.1	Anon PII	Logged-out reads of registrations/payments/push_tokens/audit_log	0 rows / 401	
20.2	Cross-tenant	O1 ↔ O2 data isolation (events, team, sponsors, analytics, check-in)	Always isolated	
20.3	Attendee ceiling	A1 cannot hit any <code>/org/*</code> or <code>/admin/*</code> route/page	Blocked server-side	
20.4	Ticket forgery	Tampered QR at check-in	Rejected (server HMAC)	
20.5	Direct API role bypass	A1 calls an admin/organizer API directly	401/403, never data	

Appendix A — Known debt (don't file these as new bugs)

- **i18n long tail:** some event-detail page literals remain English — log instances, but it's already on the post-e2e list.
- **event-detail-client.tsx** still shapes the event inline instead of using the canonical `normalizeEvent()` — refactor folded forward.
- **Broadcast rate-limit** is in-memory (per instance) — fine for the e2e; `TODO(scale)` to edge/Redis later.
- **Email/SMS/Push + Chapa payments + live AI** are intentionally stubbed/off for the MVP.

Appendix B — When something fails

Send me the section table with /  rows + notes (and a network/console error if any). I'll classify:

- **P0** (blocks the section's core flow) → I write a focused mesh handoff immediately; we fix + re-verify before moving on.
- **P1/P2** → batched into the next handoff or fold-forward; we proceed to the next section.

Each fix follows the normal loop: brain handoff → mesh builds → brain gates (`tsc/build`) + audits → merge → you re-test the affected rows.

Appendix C — First-15-minutes smoke (do this before section-by-section)

1. **2.2 JWT bridge** — log in, load "My Tickets". If it 401s, stop and tell me (legacy JWT secret issue) — nothing authed will work until that's resolved.
2. **3.1 Home** renders seeded events. **4.1** free registration issues a ticket. **8.1** check that ticket in. If those four pass, the spine is healthy and the rest is breadth.